A PROJECT REPORT

ON

POCKETPULSE: A DAILY EXPENSE TRACKER FOR ANDROID

Course: Mobile Application Development

Submitted by:

Prajapati Tirth – 12302080601123

Tapan patel – 12302080601119

Yash Patel – 12302080601134

Class: IT2 (C8A) Course / Branch: IT / Semester 7



DEPARTMENT OF INFORMATION TECHNOLOGY

A. D. PATEL INSTITUTE OF TECHNOLOGY

Constituent College of CVM University

New Vallabh Vidyanagar - 388121, Anand, Gujarat, India

Academic Year: 2026 – 2027

ABSTRACT

Managing day-to-day pocket money is a common difficulty for students and young earners. PocketPulse is a lightweight native Android application that lets users record each expense as it happens, assign it to a category, and compare spending for the day against a daily budget they set themselves.

The application is written in Java using the native Android SDK and is built with Gradle. It follows a single-screen dashboard design that shows today's date, amount spent, amount still available, a progress bar, a colour-coded warning message, a category-wise summary, and recent transactions. All data is stored locally on the device using SharedPreferences in JSON format, so the app works offline and requires no internet connection or special permissions. Money is stored internally in paise to avoid rounding errors.

The source report documents 16 test scenarios with expected results; it instructs the author to record actual pass/fail outcomes after execution. Since those outcomes are not included in the source, this report does not claim that all scenarios passed.

Keywords: Android, Java, Expense Tracker, Daily Budget, SharedPreferences, JSON, Personal Finance.

LIST OF FIGURES

Figure No.	Figure Title
Fig. 6.1	PocketPulse home/dashboard screen
Fig. 6.2	Add Expense dialog
Fig. 6.3	Dashboard showing budget exceeded state

LIST OF TABLES

Table No.	Table Title
Table 2.1	Hardware and Software Development Environment
Table 3.1	Application Architecture Layers
Table 3.2	Expense Data Model
Table 4.1	SharedPreferences Storage Schema
Table 4.2	Budget Status Logic
Table 5.1	User Interface Sections
Table 5.2	Colour Palette
Table 6.1	Functional Test Plan and Expected Results

TABLE OF CONTENTS

Abstract	2
List of Figures	3
List of Tables	4
CHAPTER 1: INTRODUCTION	6
1.1 Background & Problem Context	6
1.2 Problem Statement	6
1.3 Project Objectives	6
1.4 Scope and Deliverables	6
CHAPTER 2: TECHNOLOGY STACK & ENVIRONMENT	7
2.1 Programming Language and Platform	7
2.2 User Interface Technology	7
2.3 Local Data Storage	7

2.4 Currency Precision	7
2.5 Development Environment	7
CHAPTER 3: SYSTEM DESIGN & METHODOLOGY	8
3.1 Architecture Overview	8
3.2 Architectural Layers	8
3.3 Expense Data Model	8
3.4 Expense Categories	8
3.5 Data Flow	8
CHAPTER 4: DATABASE ENGINE & TRANSACTIONAL LOGIC	10
4.1 SharedPreferences Schema	10
4.2 Example Stored Expense	10
4.3 Input Validation	10
4.4 Exact Currency Conversion	10
4.5 Persistence and Corrupted Data	10
4.6 Budget Status Rules	10
4.7 Daily Total and Category Summary	10
CHAPTER 5: APPLICATION MODULES & INTERACTION DESIGN	12
5.1 Dashboard	12
5.2 Add Expense Dialog	12
5.3 Set Budget Dialog	12
5.4 Category Summary and Transaction History	12
5.5 Delete Confirmation	12
5.6 Visual Design	12
CHAPTER 6: RESULTS, APPLICATION WALKTHROUGH & SCREENSHOTS	14
6.1 Implementation Structure	14
6.2 Application Startup	14
6.3 Money and Display Formatting	14
6.4 Screenshots from the Source Report	14
6.5 Test Plan and Expected Results	14

6.6 Testing Considerations	14
CHAPTER 7: TRAINING EXPERIENCE, CHALLENGES & CONCLUSION	17
7.1 Limitations	17
7.2 Future Enhancements	17
7.3 Conclusion	17
REFERENCES / BIBLIOGRAPHY	18
APPENDIX	19
Appendix A: Project Directory Hierarchy	19
Appendix B: Individual Technical Responsibilities (Team of 4)	19

CHAPTER 1: INTRODUCTION

1.1 Background

Personal finance management is increasingly important as digital payments make spending quick and convenient. College students and first-time earners often make frequent small purchases—such as snacks, bus fares, stationery, or movie tickets—without keeping a reliable record. The total can accumulate unnoticed and leave less money for the remainder of the month.

PocketPulse is a lightweight native Android application intended to make daily expense tracking fast and simple. Its tagline is “Your money, under control.” Rather than offering a complex accounting workflow, the application presents a single dashboard that answers practical daily questions: how much has been spent today, how much remains within the daily limit, and where the money went.

1.2 Problem Statement

Students and first-time earners may forget paper-based records, while general-purpose finance applications can be complex, include advertisements, or require network access and permissions. A simple private application is therefore useful when it can record an expense quickly, immediately show whether spending is within the daily limit, and provide a category-wise summary of the current day.

The problem addressed by PocketPulse is the lack of a small, offline-first tool for recording individual expenses and comparing the daily total with a user-defined spending budget.

1.3 Objectives

1. Develop a native Android application that records daily expenses with a title, amount, category, optional note, and automatically assigned date and time.

2. Allow the user to set and change a personal daily spending limit.

3. Display the amount spent today, the amount remaining, and a visual progress indicator.

4. Warn the user when spending reaches 80% of the budget, reaches the budget, or exceeds it.

5. Present a category-wise summary of the current day's spending.

6. Store records locally and persistently without requiring internet access or special permissions.

7. Validate user input so that empty or invalid data is not saved.

1.4 Scope and Deliverables

In scope: a single-user, single-device Android application; daily budget tracking; six fixed expense categories; adding and deleting expenses; a transaction history; and automatic saving of data.

Out of scope for version 1.0: user accounts, cloud synchronization, multi-currency support, editing saved expenses, weekly or monthly reports, charts, and data export. The application displays amounts in Indian Rupees (₹).

The source identifies the project as a native Android application written in Java. The package name is com.example.pocketpulse and the application version is 1.0.

CHAPTER 2: TECHNOLOGY STACK & ENVIRONMENT

2.1 Programming Language and Platform

PocketPulse is implemented in Java for the native Android platform. It uses the Android SDK and a single Activity-based design. The stated minimum SDK is API 23 (Android 6.0), while compile/target SDK is API 36. The application is built using Gradle.

2.2 User Interface Technology

The interface is defined primarily in an XML layout named activity_main.xml, with dynamic views created in Java. The application uses standard Android framework components, including Activity, AlertDialog, Spinner, TextView, buttons, and other views. The source notes that no third-party libraries are used; this keeps the application small and avoids external runtime dependencies.

2.3 Local Data Storage

PocketPulse uses Android SharedPreferences for persistent key-value storage. The preferences file is named pocketpulse_prefs and stores the daily budget and the saved expense list. The expense list is serialized as a JSON array using the built-in org.json package.

Because records remain on the device, the application can operate offline. The source does not describe cloud synchronization or a remote server.

2.4 Currency Precision

Money is stored internally as whole paise rather than floating-point rupee values. Since ₹1 equals 100 paise, this representation avoids common floating-point rounding errors during additions and comparisons. BigDecimal is used when converting entered rupee amounts to paise.

2.5 Development Environment

The source report identifies Android Studio, a compatible JDK, Gradle 9.5.0, Android Gradle Plugin 9.3.3, Git, and an operating system such as Windows, macOS, or Linux as the development environment. The supplied configuration lists namespace/application ID com.example.pocketpulse, minSdk 23, compile/target SDK 36, versionCode 1, and versionName 1.0.

Table 2.1 Hardware and Software Development Environment

Component	Details
Programming language	Java
Platform	Android (native, using Android SDK)
Minimum SDK	API 23 (Android 6.0)
Target / Compile SDK	API 36
Build system	Gradle 9.5.0 with Android Gradle Plugin 9.3.3
UI definition	XML layout (activity_main.xml) plus views created in Java
Data storage	SharedPreferences (key-value storage)
Data format	JSON using the built-in org.json library
Money handling	java.math.BigDecimal and long values in paise
IDE	Android Studio
Version control	Git

The source specifies a compatible development computer and Android Studio/SDK setup; it does not provide a precise computer hardware specification.

CHAPTER 3: SYSTEM DESIGN & METHODOLOGY

3.1 Architecture Overview

The application follows a simple single-Activity architecture. MainActivity acts as the screen, contains the application logic, and manages local data storage. The report explains that the small application is kept straightforward rather than divided into a multi-layer or multi-Activity architecture.

3.2 Architectural Layers

Presentation: dashboard, dialogs, lists, and warning messages. These are defined by activity_main.xml and drawable resources, with dynamic views created by MainActivity.

Logic: budget calculations, input validation, category totals, date checking, and dashboard refresh operations are handled by methods in MainActivity.

Data: the daily budget and expense list are saved and loaded through SharedPreferences in the pocketpulse_prefs file.

Table 3.1 Application Architecture Layers

Layer	Responsibility	Where it lives
Presentation	Dashboard, dialogs, lists, warning messages	activity_main.xml, drawables, MainActivity
Logic	Budget calculation, validation, category totals, date checking	MainActivity methods
Data	Saving/loading budget and expenses	SharedPreferences (pocketpulse_prefs)

3.3 Expense Data Model

Each expense is represented by a private inner class named Expense. Its fields are: id (String), title (String), amountPaise (long), category (String), note (String), and timestamp (long). The identifier is derived from the current time in milliseconds, and the timestamp records when the expense was created.

The source describes amountPaise as the amount in paise; title as the expense name; category as one of the fixed categories; note as optional detail; and timestamp as the date and time of recording.

Table 3.2 Expense Data Model

Field	Type	Description
id	String	Unique identifier (current time in milliseconds)
title	String	Name of the expense
amountPaise	long	Amount in paise (100 paise = ₹1)
category	String	One of six fixed categories
note	String	Optional extra detail
timestamp	long	Date and time the expense was recorded

3.4 Expense Categories

The six fixed categories are Food, Transport, Education, Shopping, Entertainment, and Other. Unknown category values read from storage are normalized to Other by normalizeCategory(). The category list is fixed in version 1.0 and is not user-customizable.

3.5 Data Flow

1. On startup, onCreate() loads the layout, opens SharedPreferences, loads the budget and expense list, and calls refreshDashboard().

2. When the user selects Add Expense or Set Budget, the corresponding dialog opens.

3. The input is validated. If invalid, an error is shown and no data is saved.

4. If valid, the in-memory data is updated, written to SharedPreferences, and the dashboard is refreshed.

5. refreshDashboard() recalculates today's total, remaining budget, progress bar, warning message, category summary, and transaction list.

CHAPTER 4: DATA MANAGEMENT, VALIDATION & BUDGET LOGIC

4.1 SharedPreferences Schema

The preferences file pocketpulse_prefs stores two documented values: daily_budget_paise, a long representing the user's daily budget (default 50000 paise, or ₹500), and expenses_json, a String containing a JSON array of saved expenses.

Table 4.1 SharedPreferences Storage Schema

Key	Type	Purpose
daily_budget_paise	long	Daily budget; default 50000 paise (₹500)
expenses_json	String	JSON array containing all saved expenses

4.2 Example Stored Expense

The source gives the following illustrative stored record: {"id":"1791529457415","title":"Lunch","amountPaise":12000,"category":"Food","note":"Canteen","timestamp":1791529457415}. This is an example from the source report, not a claim that this record exists in every installation.

4.3 Input Validation

For an expense, the title must not be empty and the amount must be a valid number greater than zero. Commas are removed from the amount input before parsing. Zero, negative, or invalid text is rejected. When validation fails, an error is shown and the dialog remains open rather than closing automatically.

The daily budget must also be greater than zero. If a stored budget is zero or negative, loadBudget() resets it to the default value of ₹500.

4.4 Exact Currency Conversion

The rupee amount is normalized and converted to paise using BigDecimal, multiplying by 100 and applying scale 0 with RoundingMode.HALF_UP. The source states that invalid text returns null and is treated as an error. Storing paise as integers makes totals and comparisons exact at the supported currency precision.

4.5 Persistence and Corrupted Data

saveExpenses() converts the in-memory list into a JSON array and stores it as a string. loadExpenses() parses that JSON and validates each record: the record must have a non-empty ID and title, an amount greater than zero, and a valid timestamp. Invalid records are skipped. If the stored text cannot be parsed at all, the list is reset and a Toast message informs the user.

4.6 Budget Status Rules

The dashboard compares today's spending with the daily budget. When spending is greater than the budget, it shows “Budget exceeded by ₹X.” in red. When spending equals the budget, it shows “Daily budget reached!” in red. When spending is at least 80% of the budget but below the limit, it shows “Careful: you have used at least 80% of today's budget.” in amber. Otherwise, it shows “You are within today's budget.” in grey.

The available amount is calculated as budget minus spending but is never displayed below zero. The progress value is spent × 100 / budget, limited to a maximum of 100 for the progress bar.

Table 4.2 Budget Status Logic

Condition	Message shown	Colour
Spent > budget	Budget exceeded by ₹X.	Red
Spent = budget	Daily budget reached!	Red
Spent ≥ 80% and < budget	Careful: you have used at least 80% of today's budget.	Amber

Otherwise	You are within today's budget.	Grey

4.7 Daily Total and Category Summary

refreshDashboard() identifies today's expenses by checking the year and day of year through isToday(). It computes the daily total and remaining amount, then refreshes the warning, category summary, and transaction list.

renderCategorySummary() totals today's spending for each of the six categories using a LinkedHashMap, preserving a fixed display order. renderTransactions() creates a card for each saved expense, including title, amount, category, date and time, optional note, and a Delete button. If there are no records, the interface displays “No expenses yet. Add your first expense!”

CHAPTER 5: APPLICATION MODULES & INTERACTION DESIGN

5.1 Dashboard

The single scrollable screen displays the PocketPulse name, the tagline “Your money, under control.”, and the current date. The header uses a navy background. The Daily Budget card shows the daily limit, Set Budget action, spent-today amount, available amount, progress bar, and a warning message. A prominent Add Expense button follows the budget card.

5.2 Add Expense Dialog

The Add Expense action opens an AlertDialog with four inputs: expense title, amount in rupees, category drop-down (Spinner), and an optional note. The Save action validates the title and amount. A successful submission creates an Expense with the current time as its ID and timestamp, adds it to the list, saves the data, and refreshes the dashboard.

5.3 Set Budget Dialog

The Set Budget action displays the current budget and lets the user enter a new daily limit. The same positive-number validation is applied. The accepted value is stored under daily_budget_paise and is reflected in the dashboard.

5.4 Category Summary and Transaction History

The category summary shows six rows, one for each fixed category, with the amount spent in that category today. The Recent Transactions section is scrollable and lists saved expenses. Each entry includes a Delete button. The source does not describe a search/filter control or editing an existing transaction.

Table 5.1 User Interface Sections

Section	Contents
Header	App name "PocketPulse", tagline "Your money, under control." and today’s date on navy background
Daily Budget card	Set Budget button, daily limit, spent today, available amount, progress bar and warning
Add Expense button	Prominent "+ Add Expense" button
Today’s Category Summary	Six rows showing spending per category
Recent Transactions	Scrollable expense list with a Delete button on each entry

5.5 Delete Confirmation

Selecting Delete opens a confirmation dialog. The expense is removed only after the user confirms; cancelling keeps the record. After deletion, the data is saved and dashboard totals are recalculated, reducing the risk of accidental record loss.

5.6 Visual Design

The interface uses a Material Light theme without an action bar. Rounded cards and buttons are defined through drawable XML files. The reported palette uses Navy (#12324A) for the header and headings, Navy Dark (#0B2233) for status/navigation bars, Mint (#34C6A3) as the accent, Mint Dark (#15977B) for the available amount, Screen Background (#F4F8FA), Danger (#D64545), Warning (#B7791F), and Text Muted (#667581).

Table 5.2 Colour Palette

Name	Hex	Use
Navy	#12324A	Header and headings
Navy dark	#0B2233	Status and navigation bars
Mint	#34C6A3	Accent colour
Mint dark	#15977B	Available amount

Screen background	#F4F8FA	Page background
Danger	#D64545	Over-budget warnings, expense amounts, delete
Warning	#B7791F	80% budget warning
Text muted	#667581	Secondary text

CHAPTER 6: IMPLEMENTATION, SCREENSHOTS & TESTING

6.1 Implementation Structure

The source describes one Java class, MainActivity.java (approximately 480 lines), one XML layout, seven drawable shape files, and three resource files for colours, strings, and styles. The main methods cover application startup, expense entry, budget setting, data persistence, dashboard calculations, category summaries, transaction rendering, deletion, and formatting.

6.2 Application Startup

onCreate() sets the layout, opens SharedPreferences in private mode, binds views using findViewById(), loads saved data, and displays the dashboard. The Add Expense and Set Budget buttons are connected to their dialogs in bindViews().

6.3 Money and Display Formatting

formatMoney() converts paise to a display string such as ₹120.00. formatPlainRupees() produces a clean number for the budget input box. formatDateTime() produces a date and time string using the device's locale; the source gives an example similar to “09 Oct 2026, 12:35 PM.”

6.4 Screenshots from the Source Report

The source PDF includes three application screenshots: the home/dashboard view, the Add Expense dialog, and the dashboard after an expense exceeds the daily budget. They are reproduced in this report as source-provided visual evidence. The screenshots show a ₹500 daily budget example, an Education expense titled “Exam fee” for ₹1,500, and the resulting exceeded-budget state.

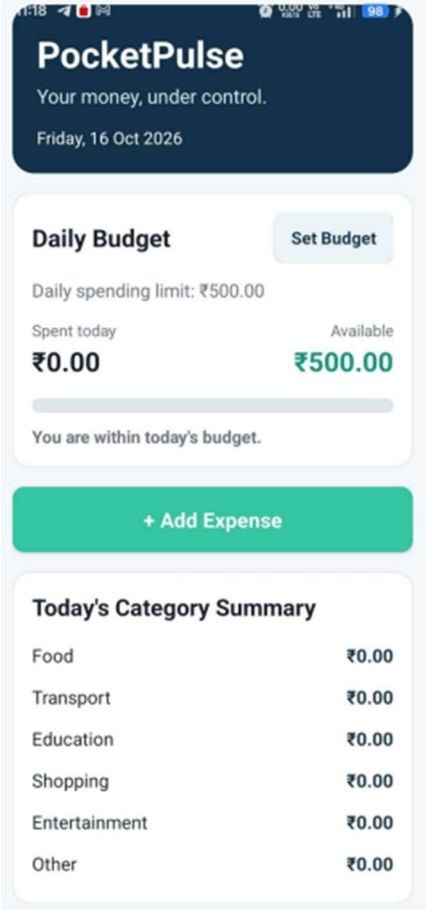


Fig. 6.1 PocketPulse application screenshot reproduced from the supplied source PDF.

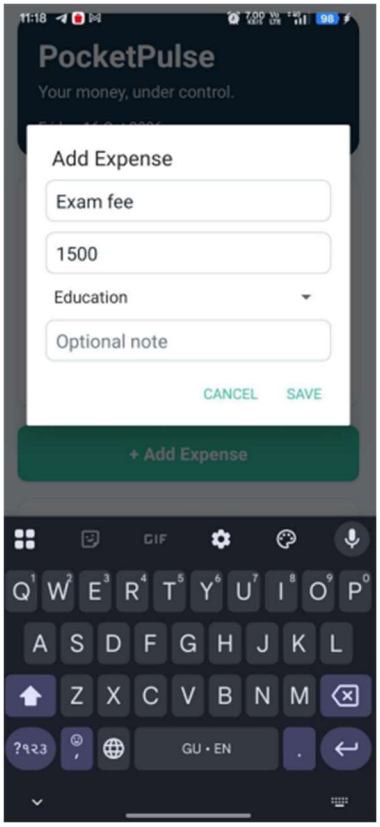


Fig. 6.2 PocketPulse application screenshot reproduced from the supplied source PDF.

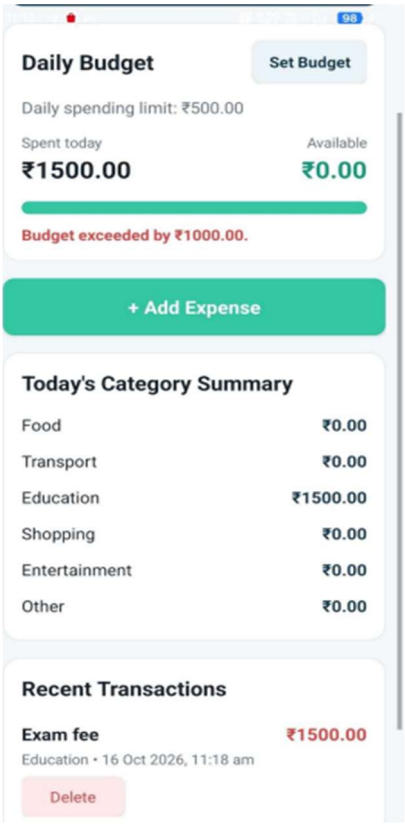


Fig. 6.3 PocketPulse application screenshot reproduced from the supplied source PDF.

6.5 Test Plan and Expected Results

The source report contains 16 test cases and explicitly instructs the author to fill in the actual result/pass-fail column after running tests on an emulator or phone. Accordingly, the table below preserves the supplied test scenarios and expected results without marking them as passed.

Table 6.1 Functional Test Plan and Expected Results

No.	Test case	Input / Action	Expected result	Actual result / Status
1	Launch app for first time	None	Dashboard opens, budget ₹500.00, spent ₹0.00, empty-state message shown	Not recorded in source
2	Add a valid expense	Title "Lunch", amount 120, category Food	Expense appears; today's spent becomes ₹120.00; Food total updates	Not recorded in source
3	Empty title	Title blank, amount 50	Error "Enter an expense title."; dialog stays open	Not recorded in source
4	Zero amount	Amount 0	Error "Enter a valid amount greater than zero."	Not recorded in source
5	Negative or non-numeric amount	-20 or abc	Same error; nothing saved	Not recorded in source
6	Decimal amount	99.50	Saved as 9950 paise; displayed as ₹99.50	Not recorded in source
7	Spend reaches 80%	Budget ₹500, spent ₹400	Amber warning displayed	Not recorded in source
8	Spend equals budget	Spent ₹500	"Daily budget reached!" in red; progress bar full	Not recorded in source
9	Spend exceeds budget	Spent ₹650	"Budget exceeded by ₹150.00." in red; available shows ₹0.00	Not recorded in source
10	Change budget	New budget 1000	Limit, available amount, progress and warning update immediately	Not recorded in source
11	Invalid budget	0 or blank	Error "Enter a valid budget greater than zero."	Not recorded in source
12	Delete an expense	Tap Delete, confirm	Expense removed; totals recalculated	Not recorded in source
13	Cancel delete	Tap Delete, then Cancel	Expense remains	Not recorded in source
14	Persistence	Close and reopen the app	Budget and expenses are still present	Not recorded in source
15	Next-day behaviour	Open app on a new day	Spent today resets to ₹0; older expenses stay in the list	Not recorded in source
16	Corrupted storage	Unreadable JSON	List is reset and a Toast message is shown; app does not crash	Not recorded in source

6.6 Testing Considerations

The listed scenarios cover initial launch, valid and invalid expense entry, decimal amounts, 80% threshold warnings, exact-budget and over-budget states, changing the budget, invalid budget input, deletion and cancellation, persistence after reopening, next-day totals, and corrupted JSON. Actual execution results are not documented in the supplied PDF.

CHAPTER 7: LIMITATIONS, FUTURE ENHANCEMENTS & CONCLUSION

7.1 Limitations

1. Saved expenses cannot be edited; they can only be deleted and entered again.

2. The budget and category summary cover only the current day; weekly and monthly reports are not available.

3. The transaction list displays all expenses together without filtering or searching.

4. The application does not include charts or graphs.

5. Data is stored only on the device. If the app is uninstalled or its data is cleared, records are lost; there is no export or backup option.

6. All expenses are stored as one JSON string. This is simple but may be unsuitable for a very large number of records because the list is rewritten after each change.

7. Categories are fixed and cannot be customized, and only Indian Rupees are supported.

8. All logic is in a single Activity class, and automated unit tests are not included.

7.2 Future Enhancements

1. Allow editing of existing expenses.

2. Add weekly and monthly budgets, reports, and date/category filters.

3. Add pie and bar charts for spending analysis.

4. Migrate from SharedPreferences to a Room (SQLite) database for better performance with larger datasets.

5. Add daily reminders and budget-alert notifications.

6. Support CSV/PDF export and cloud backup.

7. Add custom categories, income tracking, and multiple currencies.

8. Add dark mode and biometric app lock.

9. Refactor into MVVM architecture and add unit and UI tests.

7.3 Conclusion

PocketPulse is designed to provide a simple, private, and practical way to track daily spending. Users can record an expense in a few steps, immediately see how much of the daily budget remains, and receive colour-coded warnings before or after exceeding the limit. The app uses standard Android components, operates offline, stores monetary values as whole paise, and validates input and stored records to reduce errors and crashes.

Development of the project provides practical experience with the Android Activity lifecycle, XML and programmatic UI design, dialogs, input validation, JSON handling, local persistence with SharedPreferences, and precise currency calculations with BigDecimal. The source report identifies possible extensions such as reporting, charts, export, reminders, and a more modular architecture.

REFERENCES / BIBLIOGRAPHY

1. Android Developers, “Documentation for app developers”: https://developer.android.com/docs
2. Android Developers, “Save key-value data with SharedPreferences”: https://developer.android.com/training/data-storage/shared-preferences
3. Android Developers, “Dialogs” and “Layouts” guides: https://developer.android.com/develop/ui
4. Oracle, Java SE API documentation for java.math.BigDecimal: https://docs.oracle.com/en/java/javase/
5. Android API reference for the org.json package: https://developer.android.com/reference/org/json/package-summary
6. Gradle User Manual: https://docs.gradle.org

APPENDIX A: PROJECT DIRECTORY HIERARCHY

```
pocketpulse/
├── build.gradle
├── settings.gradle
├── gradle.properties
├── gradle/wrapper/gradle-wrapper.properties
└── app/
    ├── build.gradle
    └── src/main/
        ├── AndroidManifest.xml
        ├── java/com/example/pocketpulse/
        │   └── MainActivity.java
        └── res/
            ├── layout/activity_main.xml
            ├── drawable/
            │   ├── card_background.xml
            │   ├── header_background.xml
            │   ├── edit_text_background.xml
            │   ├── primary_button_background.xml
            │   ├── secondary_button_background.xml
            │   ├── delete_button_background.xml
            │   └── progress_drawable.xml
            └── values/
                ├── colors.xml
                ├── strings.xml
                └── styles.xml
```

The directory structure above reflects the paths listed in the supplied report. It is included as documentation of the reported project structure, not as a verification of every file in the source repository.

APPENDIX B: KEY CONFIGURATION (APP/BUILD.GRADLE)

Setting	Value
namespace / applicationId	com.example.pocketpulse
compileSdk / targetSdk	36
minSdk	23
versionCode / versionName	1 / 1.0